C 배열과 문자열

컴퓨터학부 박한영

목차

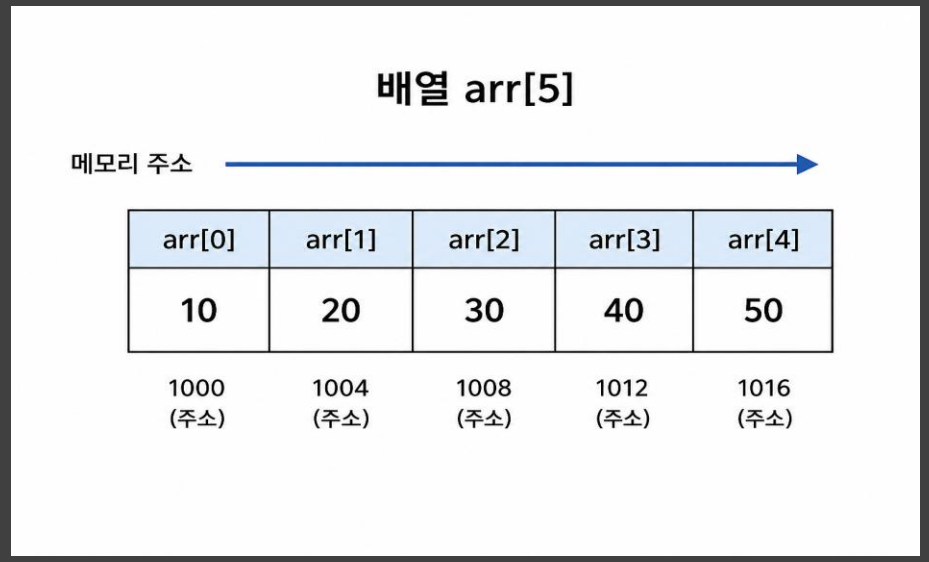
01
배열

02
문자열

배열

- 배열은 같은 자료형의 여러 변수를 하나의 이름으로 저장하기 위해 사용
- 여러 개의 데이터를 연속된 메모리 공간에 저장함
- 인덱스(index)를 이용하여 각 요소에 접근할 수 있음

```
int x1, x2, x3; // 비효율적
int x_array[3]; // 배열 사용!
x_array[0] = 1; // 인덱스를 통한 접근
```



배열 선언

-여러가지 배열 선언/초기화 방식

```
// 배열 크기 지정
int arr1[3];
// 배열 초기값 지정
int arr2[3] = {1,2,3};
// 초기값 지정 시 크기 생략 가능!
int arr2_1[] = {1,2,3};
// 나머지 부분은 0으로 채워짐
int arr2_2[5] = {1,2,3};
// 배열의 크기 없이 선언 불가능
int arr1[];
// C99 이후 변수로 크기 선언 가능
// 일반적으로는 동적 할당 사용
int n = 4;
int arr2[n];
```

컴파일러에 따라 불가능한 경우도 있으니 주의해야 함!

배열 접근

- 인덱스를 통해 배열의 요소에 접근할 수 있음

```c
int arr[5];
// 입력: 1 2 3 4 5
for(int i = 0; i < 5; i++) {
    scanf("%d", &arr[i]);
} // 출력: 1 2 3 4 5
for(int i = 0; i < 5; i++) {
    printf("%d", arr[i]);
    // 동일한 코드
    printf("%d", *(arr + i));
```

- sizeof 함수는 자료형의 메모리 크기를 바이트 단위로 반환함
- 배열의 경우, 배열 전체가 차지하는 메모리 크기를 반환함
- array == &array[0] (포인터에서 다룰 예정)

```c
// 출력: 4
printf("%d\n", sizeof(int));
int arr[30];
// 출력: 120
printf("%d\n", sizeof(arr));
```

얕은 복사 vs 깊은 복사

- 우리가 코딩을 할 때 값을 복사해야 하는 경우가 종종 생김
- 복사는 얕은 복사와 깊은 복사로 나눌 수 있음
- 얕은 복사는 복사할 대상의 주소값만 복사하므로 같은 공간을 가리킴
- 깊은 복사는 새로운 메모리 공간을 할당받고 값을 복사함

얕은 복사 vs 깊은 복사

얕은 복사와 깊은 복사의 차이

얕은 복사 (Shallow Copy)

```c
int *a = malloc(sizeof(int) * 3);
a[0] = 10; a[1] = 20; a[2] = 30;
int *b = a;     // 얕은 복사 (주소 복사)
```

[실행 결과]
b[0] = 99; 를 실행하면
a[0]도 99로 변경됨 (같은 메모리를 공유)

특징

- 주소값만 복사
- 같은 메모리 공간을 공유
- 한쪽을 수정하면 다른 쪽도 영향을 받음

깊은 복사 (Deep Copy)

```c
int *a = malloc(sizeof(int)* 3);
a[0] = 10; a[1] = 20; a[2] = 30;
int *b = malloc(sizeof(int) * 3);
for (int i = 0; i < 3; i++) {
    b[i] = a[i];    // 깊은 복사 (값 복사)
}
```

[실행 결과]
b[0] = 99; 를 실행해도
a[0]은 10 그대로 유지 (서로 다른 메모리)

특징

- 새로운 메모리를 만들고 값을 복사
- 서로 다른 메모리 공간을 가짐
- 한쪽을 수정해도 다른 쪽은 영향을 받지 않음

구분	얕은 복사 (Shallow Copy)	깊은 복사 (Deep Copy)
복사 대상	주소	실제 값
메모리 공간	공유	분리
한쪽 수정 시	다른 쪽도 영향	영향 없음
사용 예	포인터 대입	배열, 구조체, 동적 메모리 내용 복사

배열 복사

- C언어에서는 배열 전체를 대입 연산자로 복사할 수 없음
- 그래서 배열을 복사할 때는 반복문을 이용하여 일일이 복사해야 함
- 만약 포인터에 배열을 복사할 경우, 그 포인터는 원본 배열과 같은 주소 값을 가리키게 됨(얕은 복사)

```
int a[] = { 1,2,3,4,5};
int b[5];
// error!
b = a;
for(int i = 0; i < 5; i++)
    b[i] = a[i];
```

이차원 배열

- 2차원 배열은 배열 안에 배열이 존재하는 구조
- 보통 행렬(Matrix)이라고도 많이 부름
- 2차원 배열은 아래와 같이 선언할 수 있음

```
// [행의 개수][열의 개수]
int matrix1[2][3] = {{1,2,3},{4,5,6}};
// 행의 개수 생략 가능
int matrix2[][2] = {{1,1},{2,2},{3,3}};
// 1차원 배열처럼 초기화 가능
// {{1,2,3}{4,5,6}}과 동일
int matrix3[2][3] = {1,2,3,4,5,6};
```

int arr[n][m]; 의 sizeof(arr)는 4 * n * m 으로 계산할 수 있다.

문자열

- 문자열은 여러 개의 문자를 연속해서 저장한 자료형 (string)
- C언어에서 문자열은 char[] 와 같이 표현
- 문자열의 마지막에는 반드시 널 문자(\0)가 저장되어야 함
- 문자(char)는 작은 따옴표('')로, 문자열(string)은 큰 따옴표("")로 표시

```
char character = 'C';
char string[] = "Jaram";
```

문자열의 입출력

- %s 형식 지정자를 이용해 입출력을 실행함
- scanf에서 &(주소) 연산자를 사용하지 않아도 됨
- fgets() 함수를 사용하여 버퍼의 줄 단위로 문자열을 입력받을 수 있음
- 문자열 출력은 널 문자(\0)를 만날 때까지 출력하는 원리임

```
char string[30];
// scanf에서 & 사용 x
scanf("%s", string);
char buffer[30];
// fgets를 사용하면 줄 전체를 받을 수 있음!
// fgets(char *buf, int count, FILE *file);
fgets(buffer, sizeof(buffer), stdin);
printf("%s", string);
```

배열과 문자열

- char 배열과 문자열은 비슷하지만 다름!
- 배열의 경우, 끝에 null 문자를 삽입하지 않음
- 만약 문자 배열을 string으로 출력하려는 경우, 출력이 멈추지 않아 예상치 못한 결과가 발생함

```
// 문자 배열
char array[] = {'J', 'A', 'R', 'A', 'M'};
// okay
for(int i = 0; i < 5; i++)
    printf("%c", array[i]);
// error occured
printf("%s", array);
```

파싱(parsing)

- 파싱은 입력된 문자열이나 데이터를 정해진 규칙에 따라 의미 있는 단위로 나누고 해석하는 과정
- C언어에서는 string.h 헤더에서 파싱에 필요한 함수들을 지원함
- strlen, strcpy, strcmp, strcat 등의 함수들이 있음

```
// 문자열의 길이 반환
size_t strlen(const char *str);
// 문자열 dest에 src를 복사함
char *strcpy(char *dest, char *src);
// str1과 str2가 같은 지 비교함
int strcmp(const char *str1, const char *str2);
// dest의 뒤에 origin을 추가함
char* strcat(char *dest, const char *origin);
```

THANK YOU!